A Practical Guide to Fast, Concurrent Python

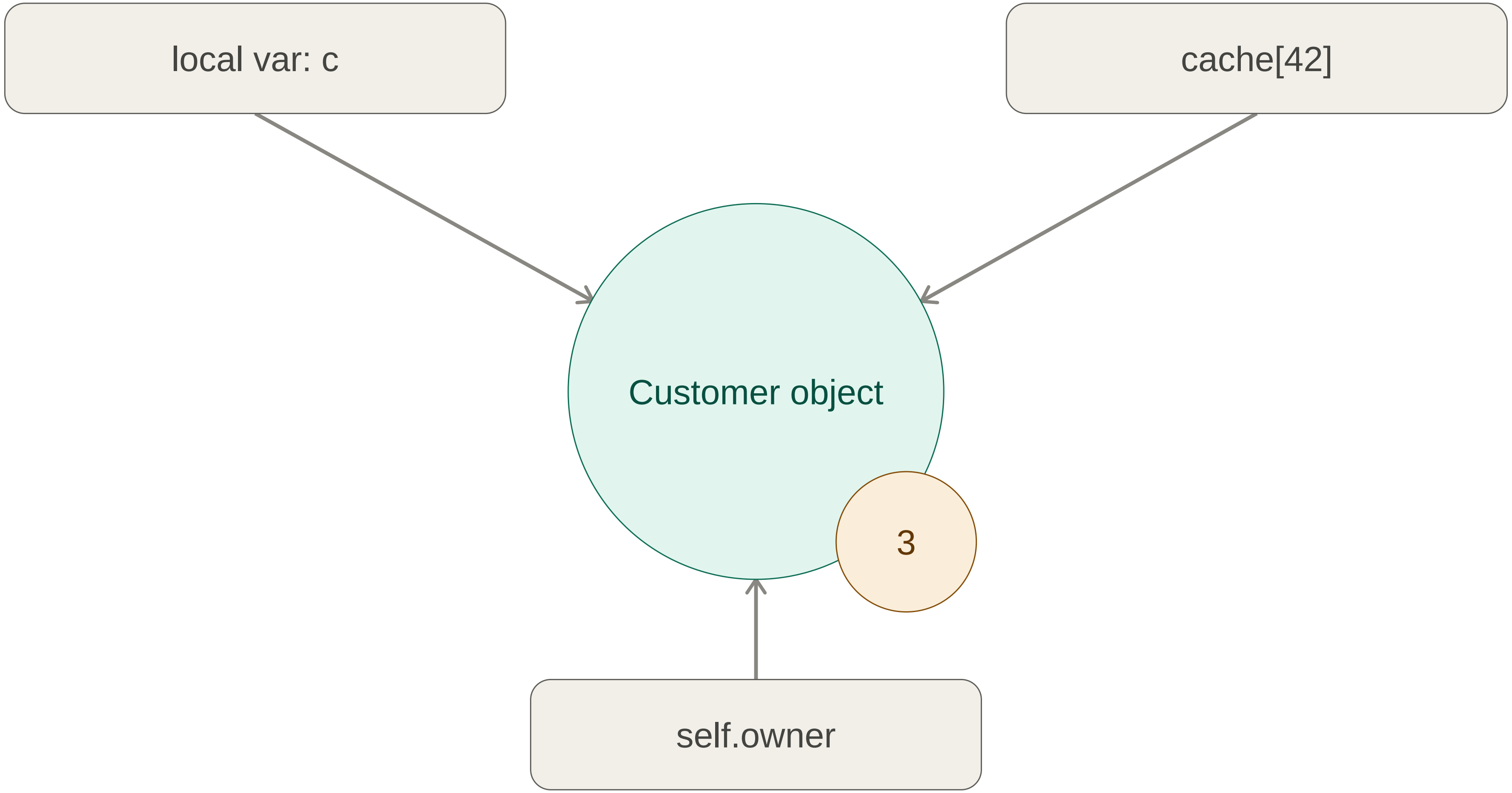
Andrzej Niedziółka

Agenda:

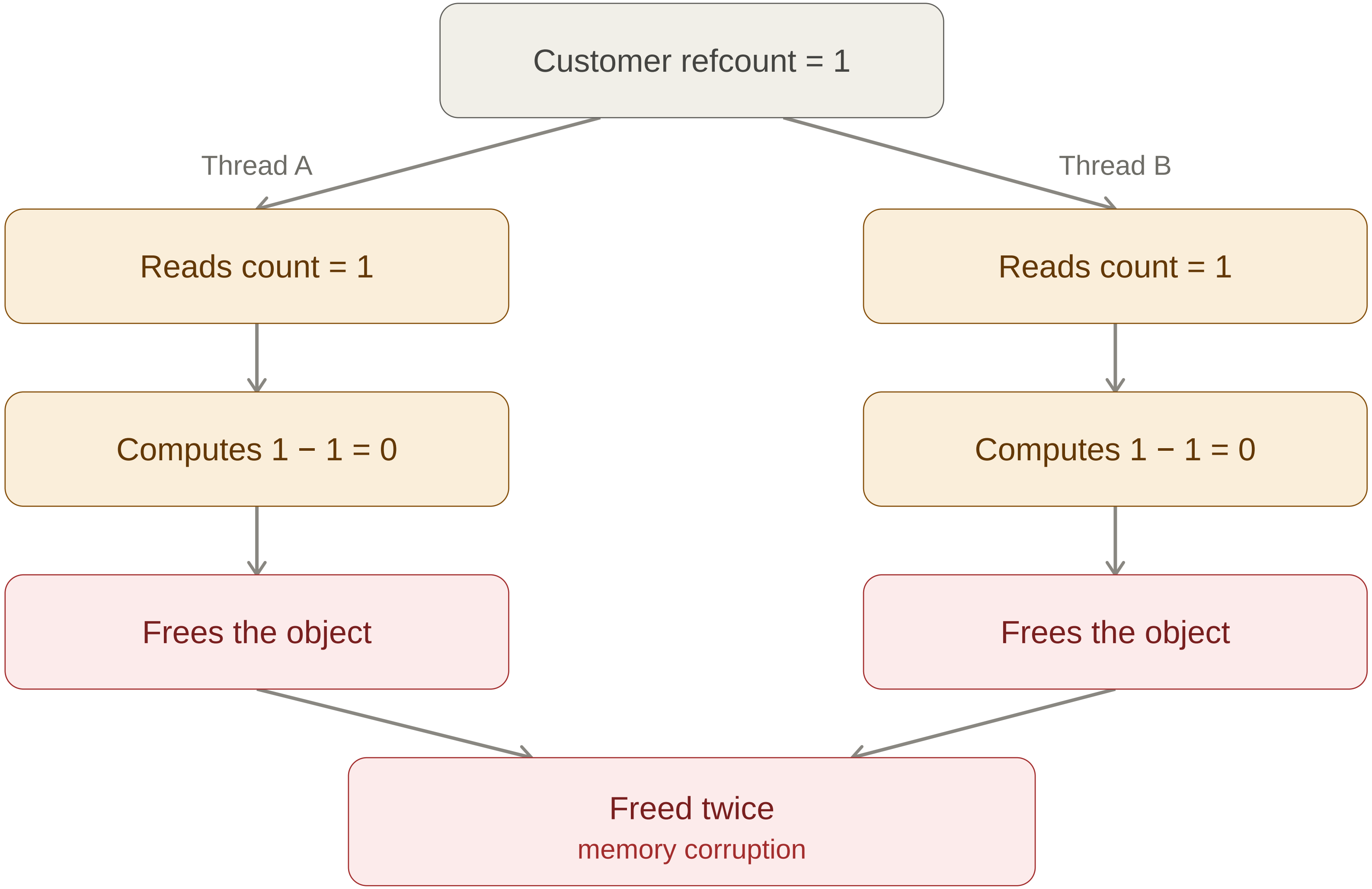
- GIL (Global interpreter lock) - what is it and how it came to life
- CPU vs I/O bound problems
- Threads vs Processes
- Locks, Semaphores
- asyncio
- No-GIL Python 3.14

Why do we need GIL?

Reference counting - Primary garbage collection mechanism



Race condition it causes

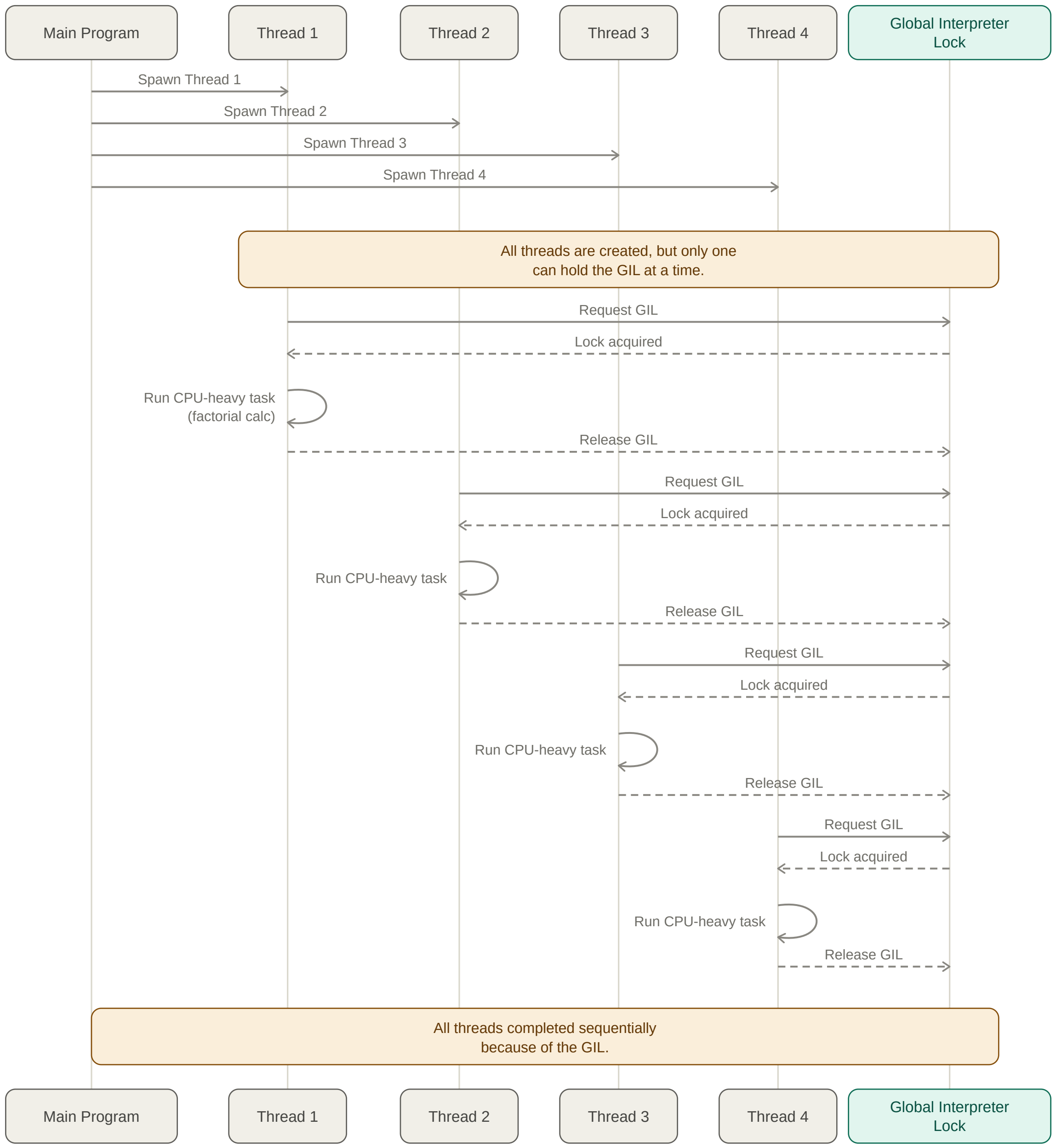


Locks to the rescue

```
import threading

counter = 0
lock = threading.Lock()

def increment():
    global counter
    for _ in range(100_000):
        with lock:  # only one thread can be inside this block at a time
            counter += 1

threads = [threading.Thread(target=increment) for _ in range(4)]
for t in threads:
    t.start()
for t in threads:
    t.join()

print(counter)  # always 400_000 — without the lock, this is racy and comes out lower
```

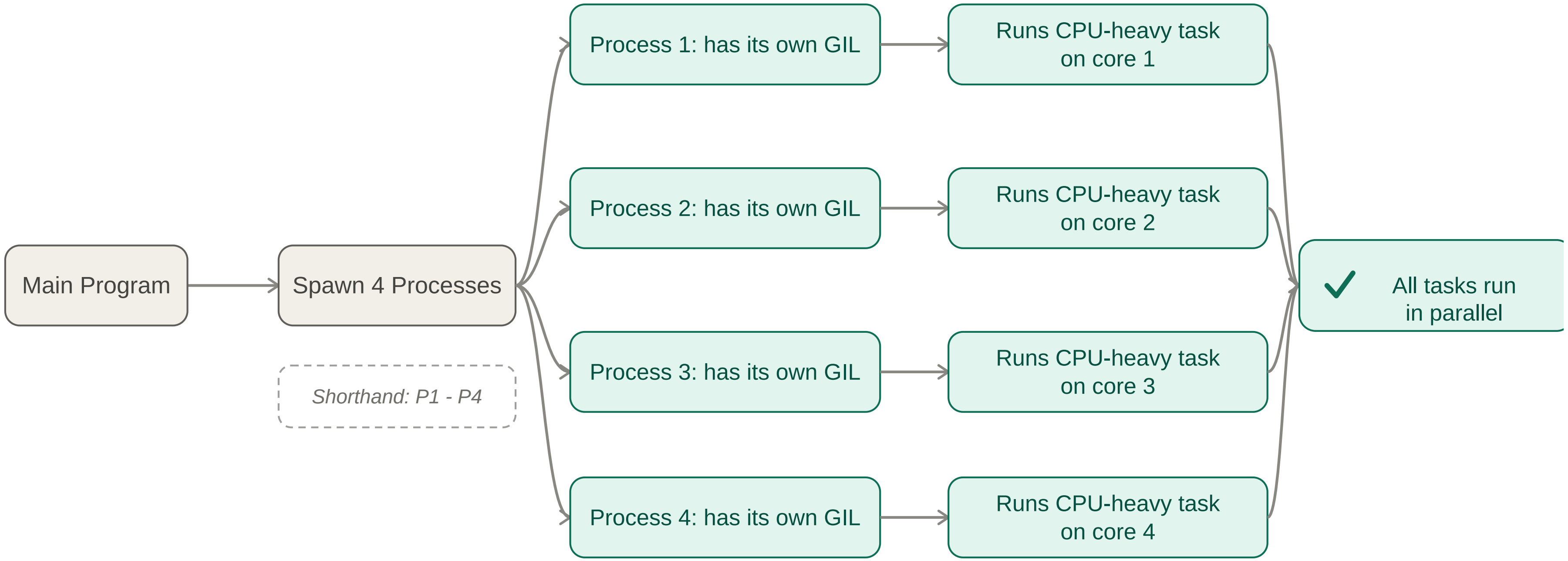
GIL (Global interpreter lock)

- A thread must acquire the GIL before running any bytecode; if another thread holds it, the requesting thread waits.
- The GIL is released in two scenarios:
  - when a thread executes a blocking I/O operation (network requests, file reads), letting other threads proceed
  - after a fixed switching interval (default 5ms), which forces a context switch to the next waiting thread


Main Program
Thread 1
Thread 2
Thread 3
Thread 4
Global Interpreter Lock
Spawn Thread 1
Spawn Thread 2
Spawn Thread 3
Spawn Thread 4
All threads are created, but only one can hold the GIL at a time.
Request GIL
Lock acquired
Run CPU-heavy task (factorial calc)
Release GIL
Request GIL
Lock acquired
Run CPU-heavy task
Release GIL
Request GIL
Lock acquired
Run CPU-heavy task
Release GIL
Request GIL
Lock acquired
Run CPU-heavy task
Release GIL
All threads completed sequentially because of the GIL.

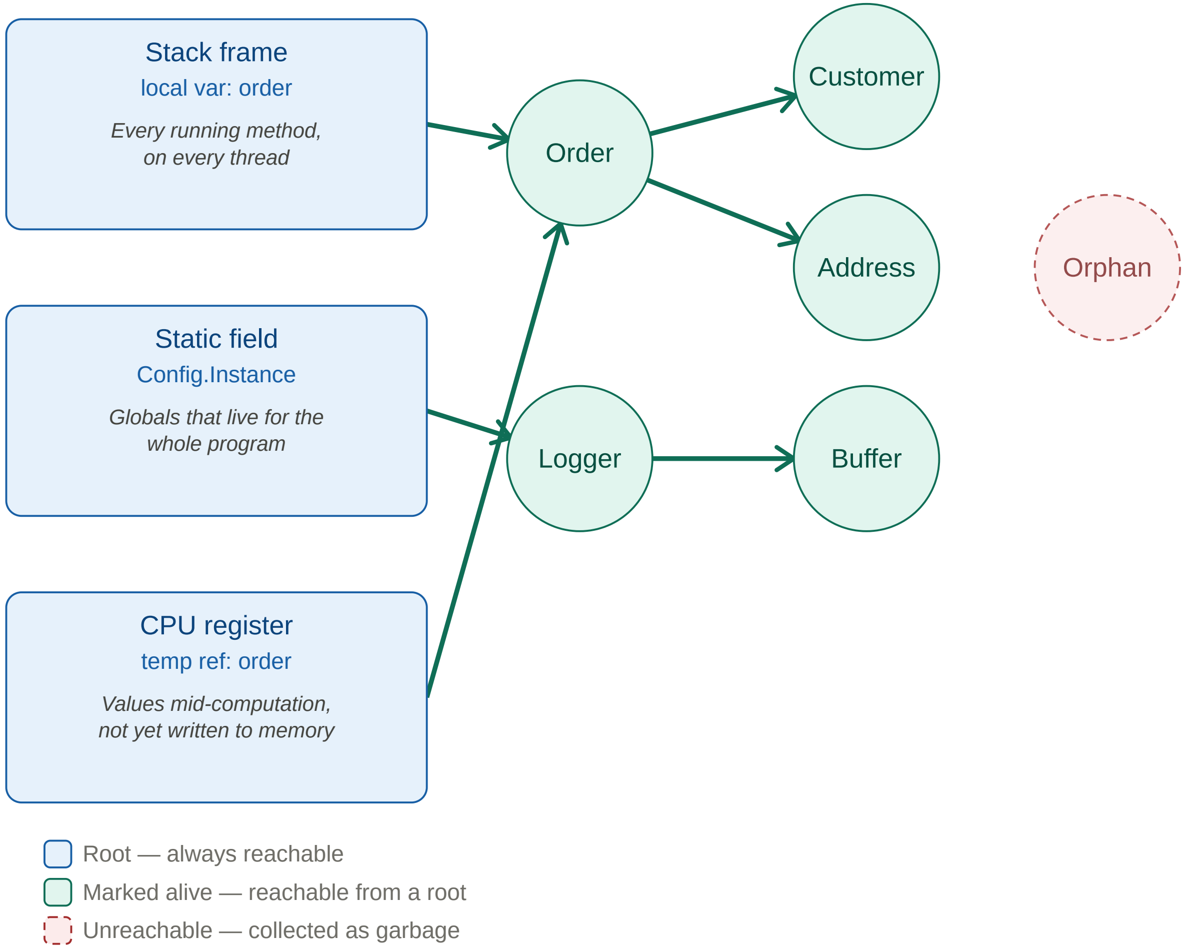
Multiprocessing sidesteps the GIL

Each process is a separate Python interpreter with its own GIL, so CPU-bound tasks can run in true parallel across multiple cores:



Why other languages may not need GIL?

Most other managed languages (C#, Java, etc.) use a **tracing garbage collector**.



Types of Performance Problems

I/O-bound vs CPU-bound

- **I/O-bound**: API calls, DB queries, file/network reads
- **CPU-bound**: image resizing, number crunching, parsing/serialization

Performance Problem Strategies

I/O-bound problems:

- Parallelize using threads - cheaper and faster than processes
- Cache heavily

CPU-bound problems:

- Because of GIL, spawning multiple threads won't help
- 1st strategy: **optimize** the code itself
- 2nd strategy: parallelize using **processes**

I/O - Multithreading

Simple example

```python
from concurrent.futures import ThreadPoolExecutor
import requests

def fetch_url(url):
    response = requests.get(url)
    return response.status_code

urls = get_urls_to_fetch()

with ThreadPoolExecutor(max_workers=len(urls)) as executor:
    futures = [executor.submit(fetch_url, url) for url in urls]
    results = [f.result() for f in futures]
    print(results)
```

Sizing ThreadPoolExecutor

```
urls = get_urls_to_fetch()  # say this returns 37 URLs

with ThreadPoolExecutor(max_workers=len(urls)) as executor:
    futures = [executor.submit(fetch_url, url) for url in urls]
    results = [f.result() for f in futures]
    print(results)
```

Semaphores

Lock generalized to N permits — bounding concurrency (e.g. "only 5 requests at once")

```python
import requests
from pathlib import Path

session = requests.Session()

def fetch_repo(full_name: str) -> int:
    r = session.get(f"https://api.github.com/repos/{full_name}", timeout=10)
    return r.json()["stargazers_count"]

def fetch_user(login: str) -> int:
    r = session.get(f"https://api.github.com/users/{login}", timeout=10)
    return r.json()["followers"]

if __name__ == "__main__":
    repos = Path("data/repos.txt").read_text().split()
    users = Path("data/users.txt").read_text().split()
```

Scenario 1: As many threads as possible

```python
from concurrent.futures import ThreadPoolExecutor

# ... inside if __name__ == "__main__":

# One thread per name, no cap — fires every request at once, regardless
# of what GitHub actually allows
with ThreadPoolExecutor(max_workers=len(repos) + len(users)) as executor:
    futures = [executor.submit(fetch_repo, name) for name in repos]
    futures += [executor.submit(fetch_user, login) for login in users]
    results = [f.result() for f in futures]
    # requests.exceptions.HTTPError: 403 Client Error: rate limit exceeded
    # — unauthenticated that's 60 requests, so it fails almost immediately
```

Scenario 2: Retry when we get told off

```python
import time
import requests

def call_with_retry(fn, arg, max_attempts=5):
    for attempt in range(max_attempts):
        try:
            return fn(arg)
        except requests.HTTPError as e:
            if e.response.status_code not in (403, 429):
                raise
            time.sleep(int(e.response.headers.get("retry-after", 2 ** attempt)))
    raise RuntimeError(f"{fn.__name__} still limited after {max_attempts} attempts")

with ThreadPoolExecutor(max_workers=len(repos) + len(users)) as executor:
    futures = [executor.submit(call_with_retry, fetch_repo, n) for n in repos]
    futures += [executor.submit(call_with_retry, fetch_user, u) for u in users]
    results = [f.result() for f in futures]
```

Scenario 3: Size a semaphore to the documented limit

```python
import threading

# GitHub documents "no more than 100 concurrent requests" — stay well under it.
# One semaphore shared by both operations, because the limit is per account.
MAX_CONCURRENT = 10
semaphore = threading.Semaphore(MAX_CONCURRENT)

def call_limited(fn, arg):
    with semaphore:  # the 11th caller blocks here until a permit frees up
        return call_with_retry(fn, arg)

with ThreadPoolExecutor(max_workers=len(repos) + len(users)) as executor:
    futures = [executor.submit(call_limited, fetch_repo, n) for n in repos]
    futures += [executor.submit(call_limited, fetch_user, u) for u in users]
    results = [f.result() for f in futures]
```

The semaphore does not replace the retry — it composes with it.

CPU - Optimizing the code

Before reaching for threads or processes, it's often cheaper — and simpler — to just make the single-threaded code faster:

Optimization 1: Set/Dict lookups

Swap list membership checks for set/dict lookups — `x in my_list` is O(n), `x in my_set` is O(1):

```
needle = "z"
# O(n) per check
allowed = ["a", "b", "c"]
if needle in allowed:
    ...

# O(1) per check
allowed = {"a", "b", "c"}
if needle in allowed:
    ...
```

Optimization 2: `io.StringIO` for building strings in a loop

Strings are immutable, so repeated `+=` is quadratic. `io.StringIO` uses a mutable buffer:

```
import io

# Quadratic — each += allocates new string
result = ""
for chunk in chunks:
    result += chunk

# Linear — writes into buffer
buf = io.StringIO()
for chunk in chunks:
    buf.write(chunk)
result = buf.getvalue()
```

Optimization 3: Reach for libraries that drop into C under the hood

NumPy does number crunching in compiled C over contiguous memory:

```python
import numpy as np

# Pure Python — per-element bytecode dispatch
squares = [x * x for x in range(1_000_000)]

# NumPy — runs once in C
squares = np.arange(1_000_000) ** 2
```

CPU - Parallelize using processes

```python
from concurrent.futures import ProcessPoolExecutor

def crunch_numbers(n):
    return sum(i * i for i in range(n))

with ProcessPoolExecutor(max_workers=4) as executor:
    results = list(
        executor.map(crunch_numbers, [10_000_000] * 10)
    )
```

Sizing ProcessPoolExecutor

```
import os
from concurrent.futures import ProcessPoolExecutor

def crunch_numbers(n):
    return sum(i * i for i in range(n))  # simulate CPU-bound work

# Unlike threads, more processes than CPU cores doesn't buy more
# parallelism — they'd just compete for the same cores.
# os.cpu_count() counts logical cores, so hyperthreaded siblings are
# included — for CPU-bound work they're worth well under a full core each.
available_cores = os.cpu_count() or 1
worker_count = max(1, available_cores - 1)  # leave a core for the OS/main process

with ProcessPoolExecutor(max_workers=worker_count) as executor:
    results = list(executor.map(crunch_numbers, [10_000_000] * 10))
```

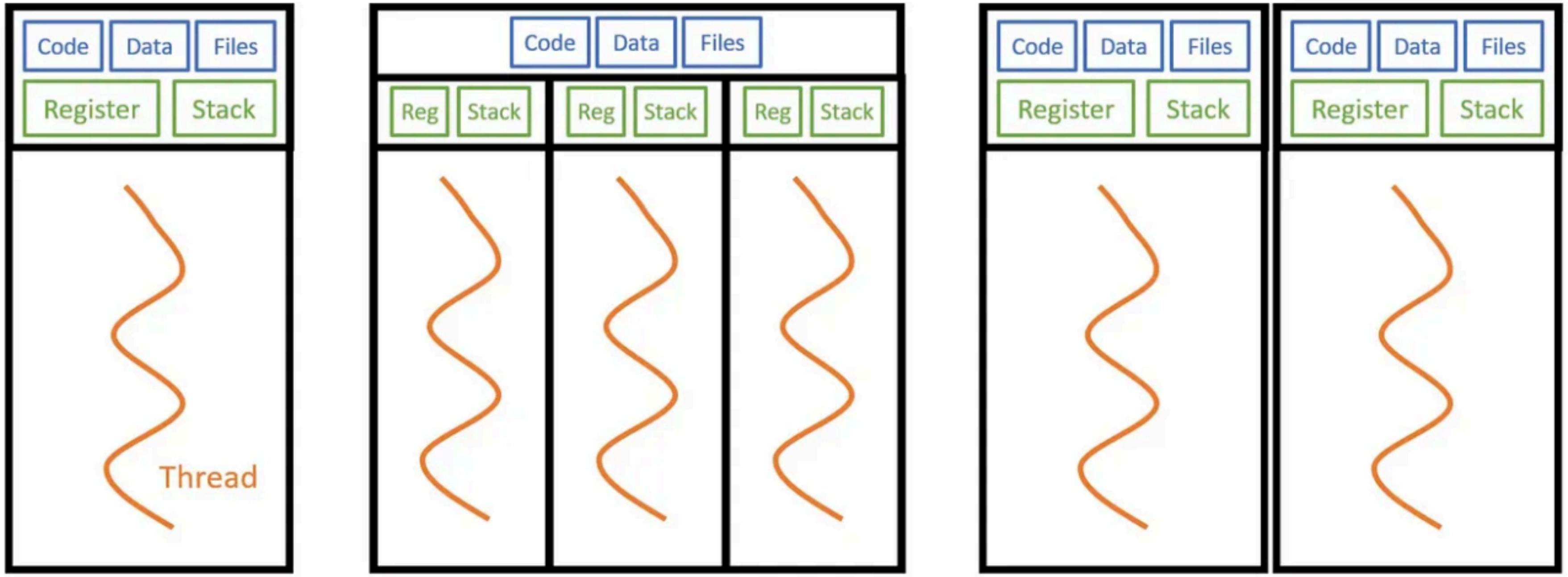
Kubernetes: Pass CPU cores as environment variable

```yaml
env:
  - name: CPU_LIMIT
    valueFrom:
      resourceFieldRef:
        resource: limits.cpu
        divisor: "1"      # rounds fractional limits UP to whole cores
```

Spawning more processes increases RAM usage — be careful not to run out of memory.

Threads vs Processes summary

Feature	Threads	Processes
Memory	Shared within the same process	Separate, isolated memory spaces
Best For	I/O-bound tasks	CPU-bound tasks
Execution	Concurrency (context switching)	Parallelism (true simultaneous execution)
GIL Impact	Limited by single GIL per process	Each process has its own GIL
Overhead	Low (lightweight)	High (heavier resource usage)
Communication	Direct memory access (easier)	Pipes/queues (more complex)
Robustness	Lower (crash affects whole process)	Higher (isolation prevents cascade failure)



Single Processor Single Thread Single Processor Multithread Multiprocessing

asyncio

- **async/await** syntax for concurrent code
- Great for **high-volume I/O**
- **Single-threaded** event loop (no switching overhead)

Simple asyncio example

```python
import asyncio
import httpx

async def fetch_url(client, url):
    response = await client.get(url)
    return response.status_code

async def main():
    urls = get_urls_to_fetch()
    async with httpx.AsyncClient() as client:
        return await asyncio.gather(
            *(fetch_url(client, u) for u in urls))

asyncio.run(main())
```

asyncio with semaphore

Same idea as threads — only the primitive changes:

```python
import asyncio
import httpx

MAX_CONCURRENT = 10
semaphore = asyncio.Semaphore(MAX_CONCURRENT)

async def fetch_limited(client, url):
    async with semaphore:
        return await fetch_url(client, url)

async def main():
    urls = get_urls_to_fetch()
    async with httpx.AsyncClient() as client:
        return await asyncio.gather(
            *(fetch_limited(client, u) for u in urls))

asyncio.run(main())
```

Threads vs asyncio

With threads, `max_workers` is both cap and budget. With asyncio, only the semaphore caps concurrency (tasks are nearly free).

	Threads	**asyncio**
Concurrency unit	OS thread (MBs of stack)	coroutine (bytes on the heap)
Switching	done by the kernel	managed inside python code
Practical ceiling	hundreds → low thousands	tens of thousands+
Works with sync libraries	yes, unchanged	no — needs async-native clients
Blocking call impact	that thread only	stalls *every* task
CPU-bound work	still GIL-limited	still GIL-limited (use processes)

Free-Threaded Python (3.14)

Two refcount fields per object

Thread that owns object updates *local* count (no locking). Other threads touch *shared* count. True refcount = local + shared.

Result: **no locking and no atomic instructions** for the common case.

Per-object locks for containers

Each `dict`, `list`, `set` has its own fine-grained lock. Two threads mutating different dicts never contend.

GC pauses

Garbage collection now does **two brief stop-the-world pauses** (unlike classic GIL-protected collection).

Dependency requirements

Any C extension not explicitly marked thread-safe **re-enables the GIL for the whole process** when imported.

Impact

Free-threading makes `ThreadPoolExecutor` the right answer for **both** CPU-bound and I/O-bound workloads.

Thank You!

Questions?